

Late Breaking Results: Versatile 4:1 Multiplexer Using 1T1R RRAM Crossbar for High Speed In-Memory Computing

Vinodh Kumar B, *Student Member, IEEE*, Binsu J Kailath, *Member, IEEE*

Department of Electronics and Communication Engineering

Indian Institute of Information Technology, Design and Manufacturing, (IIITDM) Kancheepuram

Abstract—This paper presents a high speed NAND based 4:1 Multiplexer (MUX) for various logic operations in a Resistive-Random access memory crossbar structure. Compared to existing 4:1 MUX designs that require 10 steps, our proposed method completes execution in 3 steps. Additionally, proposed 4:1 MUX based Full adder, Full subtractor and 1-bit Comparator reduce clock cycles by 75% compared to state-of-the-art. Despite an increase in memristor count, the reduction in computational delay makes this approach promising for fast in-memory computing.

Index Terms—Memristor, Non-Volatile Memory, Multiplexing, Logic functions, Logic-in-memory

I. INTRODUCTION

In order to realize In-Memory Computing (IMC) using Resistive-Random access memory (RRAM) which stores information in resistance states as low resistance (LRS) and high resistance (HRS), realizing crossbar architecture without additional mapping circuitry has been a challenge addressed by the researchers [1]. Implementing analog and digital computations using RRAM requires dedicated mapping and exclusive control blocks, adding to system complexity.

To overcome these challenges, this paper proposes a versatile NAND based multiplexer which doesn't require additional mapping circuitry for implementing multi-functional logic blocks. This approach has been validated by implementing 4:1 MUX, Full adder (FA), Full subtractor (FS) and 2-bit comparator and comparing the results with the start-of-the-art [2] - [4]. To the best of our knowledge, this is the first multi-functional RRAM crossbar structure which doesn't require additional mapping and control circuitry which compute the result in just three clock cycles.

II. PROPOSED DESIGN

A. 1T1R RRAM Structure

The 1T1R (one-transistor, one-resistor) structure has been realized using a VTEAM memristor [5] and gpdn 45 nm MOSFET model. The set and reset voltages of the memristor have been optimized to +1.65 V and -0.35 V respectively for 1T1R structure resulting in a resistances range of 1 k Ω to 300 k Ω . Resistance transitions occur only when the gate voltage of the MOSFET is enabled; otherwise, the memristor retains its state, regardless of set/reset conditions.

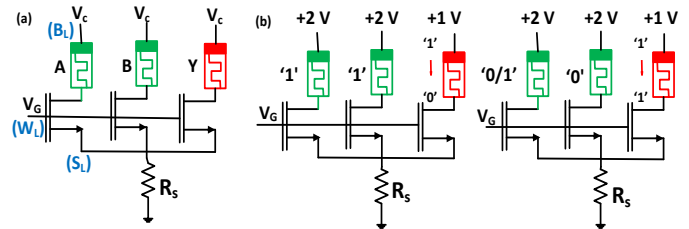


Fig. 1. NAND logic [6]. (a) Basic circuit, (b) LRS ('0') & HRS ('1') are initiated to inputs (A,B) and output is set to HRS. The control voltage is applied as 2 V to inputs and 1 V to output, R_s is chosen as 10 k Ω .

The word line (WL) enable/disable the transistor gate while the bit line (BL) connect to the memristor's top electrode where the control voltage is applied. The state of the memristor is detected by measuring the current pumped out of the source line (SL) which is found to be between 6 μ A and 660 nA during set and reset conditions.

B. Stateful NAND Logic Implementation

Three 1T1R [6] structures have been horizontally connected with the same gate-source voltage wherein all gates are connected to enable/disable voltage V_G and all sources share common source resistance R_s , set between LRS and HRS as shown in Fig. 1.

Here, the first two memristor's serve as inputs A and B, while the third acts as output Y, initially set to HRS ('1'). The input memristors receive a higher control voltage than that at the output, creating a voltage drop across R_s . This drop facilitates the output memristor's transition from HRS. Once the control voltage is applied, the output state depends on the input memristors, replicating NAND gate logic.

C. Proposed 4:1 MUX using 5*5 1T1R crossbar Array

The proposed NAND-based 4:1 MUX is implemented using a 5*5 crossbar array, as shown in Fig. 2(a) and Fig. 3(a). This design is optimized for vertical execution and computing process is explained below. Achieving both vertical and horizontal execution is possible but requires additional isolation transistors which will increase the circuit complexity.

1) *Initialization and Parallel Execution*: Initially, all input signals (I_0, I_1, I_2, I_3) and selection lines (S_0, S_1) are programmed into the memristor array using set/reset voltages. The initial outputs (X_1, X_2, X_3, X_4) are set to HRS ('1') by

default. A control voltage of +2 V is applied to the first three rows (BL₁, BL₂, BL₃) while the fourth row (BL₄) receives +1 V. This potential difference across the output memristors induces a resistance state transition based on the input state, enabling parallel execution across the first four columns.

2) *Sense Amplification and Peripheral Processing*: In the next stage, a small read voltage (+0.2 V) is applied to BL₄, allowing the sense amplifier (SA) to measure the output currents from memristors (X₁ - X₄) via the SL. The SA detects two distinct current levels: 6 μ A for HRS ('1') and 660 nA for LRS ('0'). The read and write peripheral units process these sensed currents, converting 6 μ A into a set voltage and 660 nA into a reset voltage, which are then applied to the memristors in the fifth column.

3) *Final Computation and Output Generation*: In the final stage, the last column undergoes the same execution process as the first stage, leading to a resistance transition in the output memristor (Y) and the corresponding change in current through R_S. Different possible combinations of input lines (I₀, I₁, I₂, I₃) and select lines (S₀, S₁) and corresponding resistance transition in the memristor are illustrated in Fig. 2(b). The transition in current for input combinations 1011 and 1010 are illustrated in Fig. 3(b) for S₁S₀ = 00. The above execution for realizing a 4:1 MUX takes four clock cycles for initialization and three clock cycles for computation.

D. Realization of multiple Arithmetic and logic function using proposed 4:1 MUX

Full adder (FA), Full Subtractor (FS) and 1-bit Comparator have been realized with proposed crossbar architecture. Truth table following the conventional MUX based logic implementation for the above circuits are given in Fig. 4, where A, B, C represents the input bits.

III. RESULTS AND DISCUSSION

Simulations have been performed in Cadence virtuoso for all the above arithmetic and logic blocks and results for MUX, FA, FS and Comparator are summarized in Table I. Memristor count and number of clock cycles are compared with the state-of-the-art works and it can be observed that proposed architecture completes the computation in three clock cycles. Higher count in memristor required for the proposed design is justified as it doesn't require additional circuitry and performs all the above logic functions with same number of memristors.

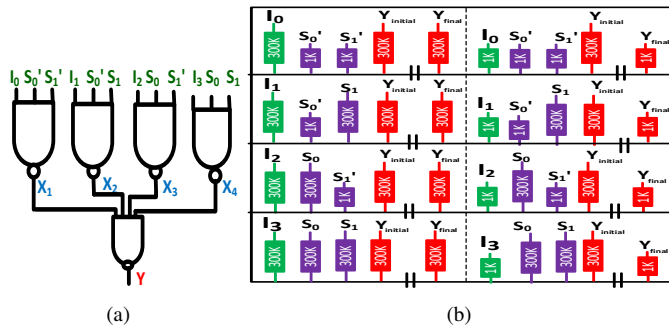


Fig. 2. (a) NAND Based 4:1 MUX (b) Output resistance transition for all input combination.

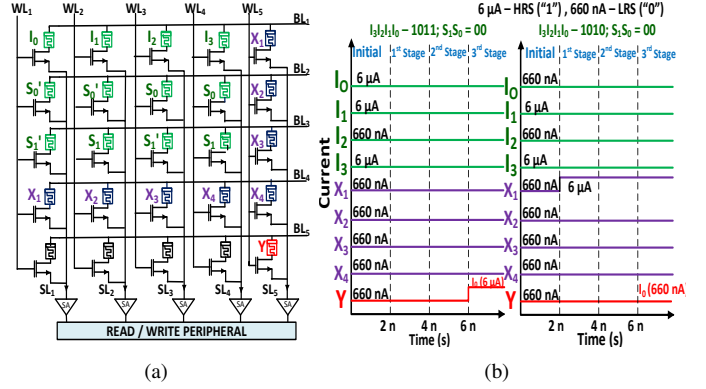


Fig. 3. (a) Crossbar Structure of 4:1 MUX (b) Illustration of Current flow across memristor during execution.

Function	S ₁	S ₀	I ₀	I ₁	I ₂	I ₃
Half Adder Sum	A	B	0	1	1	0
Half Adder Carry	A	B	0	0	0	1
Full Adder Sum	A	B	C	C	C	C
Full Adder Carry	A	B	0	C	C	0
Half Subtractor Diff	A	B	0	1	1	0
Half Subtractor Borrow	A	B	0	1	0	0
Full Subtractor Diff	A	B	C	C	C	C
Full Subtractor Borrow	A	B	C	1	0	C
Even Parity Generator	A	B	1	0	0	1
Odd Parity Generator	A	B	0	1	1	0
< Comparator	A	B	0	1	0	0
> Comparator	A	B	0	0	1	0
= Comparator	A	B	1	0	0	1

TABLE I
COMPARISON WITH EXISTING WORKS

Ref.	Implementation	Memristor Count	Clock Cycle
[2]	Full Adder (FA)	15	12
[3]	Full Subtractor (FS)	30	12
[3]	1-bit Comparator	19	12
[4]	4:1 MUX	7	10
This Work	4:1 MUX	25	3
This Work	Full Adder	50	3
This Work	Full Subtractor	50	3
This Work	1-bit comparator	25	3

Fig. 4. Logical function implemented using 4:1 MUX where A, B and C are inputs for corresponding logic. * There is no initialization steps included.

IV. CONCLUSION

The proposed memristor crossbar structure eliminates the need for separate mapping and control circuits resulting in efficient execution of logic operations. Significant reduction in clock cycles while implementing multiple logic make it adaptable to broader computational requirements for in-memory computing.

V. ACKNOWLEDGMENT

This work was supported by the Visvesvaraya PhD and C2S Schemes (MeitY) and SPARC Program (MoE), Govt. of India. The authors would also like to thank Dr. R.V. Joshi (IBM, USA) and Prof. Yu Cao (University of Minnesota, USA) for their valuable insights and support.

REFERENCES

- [1] G. Krishnan et al., "Design Limits of In-Memory Computing: Beyond the Crossbar," 2021 5th IEEE Electron Devices Technology and Manufacturing Conference (EDTM), Chengdu, China, pp. 1-3, May 2021.
- [2] Jiang, M et al. An efficient memristive alternating crossbar array and the design of full adder. Nonlinear Dynamics, Volume 111, 20331–20345, September 2023.
- [3] GN. Kaushik et al., "Energy Efficient Memristor-Based Subtractors and Comparator for In-Memory Computing in MAGIC," in IEEE Transactions on Circuits and Systems II, no. 6, pp. 3171-3175, June 2024.
- [4] C. Zhang et al., "Logic-in-Memory Approach Realizing Memristor-Based Multiplexers," in IEEE Transactions on Electron Devices, vol. 70, no. 12, pp. 6341-6346, December 2023.
- [5] S. Kvatinisky et al., "VTEAM: A General Model for Voltage-Controlled Memristors," in IEEE Transactions on Circuits and Systems II: Express Briefs, vol. 62, no. 8, pp. 786-790, August 2015.
- [6] W. Shen et al., "Stateful Logic Operations in One-Transistor-One-Resistor Resistive Random Access Memory Array," in IEEE Electron Device Letters, vol. 40, no. 9, pp. 1538-1541, September 2019.